

L01

09:00 - 16:00, 16:15 - 17:00 freiwillig

Grading:

- Quiz, 11pts per event, 60% threshold -> 6.6pts, letzte halbe Stunde
- Extra-Punkte: Toolbox bauen -> Projekt, bis zu 10 extra Projekte, Repo schicken, (.md File hinzufügen)

document.writeln("something"): to write onto page

JavaScript funtions:

- JavaScript doesnt need a Compiler
- 2 phase interpreter: first phase only locates space for variables and object, second phase initialises
- cant overload method

- `report()` outputs results to the page using `document.writeln(...)` (it prints each evaluated expression).
- JavaScript functions don't enforce argument counts.
 - `fun1()` returns `1`.
 - Calling `fun1(42)` still returns `1` because extra arguments are simply ignored unless you use them.
- Function declarations with the same name get overwritten ("last one wins").
 - `fun2()` is defined twice; the second `fun2(arg) { return arg; }` replaces the first.
 - So `fun2()` returns `undefined` (because `arg` is missing), not `1`.
- If you don't explicitly `return`, you get `undefined`.
 - `noReturn()` and `noReturn2()` both return `undefined` because they have no `return`.
 - `noReturn3 = () => 1` does return `1` because it's an expression-bodied arrow function.
- Default parameters work as a fallback.
 - `noReturnDefArg(argo = 0)` returns `0` when called with no argument, otherwise returns the provided value.
- Functions are first-class values.
 - You can assign them (`const myfun = fun1`) and store them in arrays (`funs = [null, undefined, fun1, fun2]`) and call them via references.
- Currying / higher-order functions.
 - `doit` and `doit2` take a function (`whatToDo`) and return a new function that calls it with an argument.
 - This allows calls like `doit(fun2)(10) → 10`, and `doit(fun1)(10) → 1`.
 - `doFun2 = doit2(fun1)` "preloads" `fun1`, so `doFun2(10)` and even `doFun2()` both return `1` (since `fun1` ignores arguments anyway).

hoisted: can be used before declaration like function, const is not hoisted

- three types of string: "it's time" and 'he said "hi"' and `x ist \${x}`
`String(/hdghsj/)`
`typeof String = function`

- two scopes:
global: alles zugreifbar (im browser: window)
function: variables are local to the enclosed function

`let x = ...` mutable, local scope

`const x = ...` immutable*, local scope

- IIFE: immediately invoked function expression
`(() => {
 let x = 1;
 document.writeln(String(x));`

Beta Reduktion II

$(f \rightarrow x \rightarrow f(x)) (id)(1)$

- in L. immediately invoked function expression

```
((() => {
    let x = 1;
    document.writeln(String(x));
})());
```

- Lambda Calculus

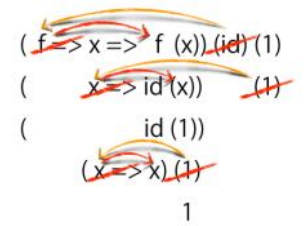
alpha: renames variables, $x \Rightarrow x$ is alpha equivalent to $y \Rightarrow y$

beta: runs function, $(x \Rightarrow x+1)(5)$, beta reduces to 6

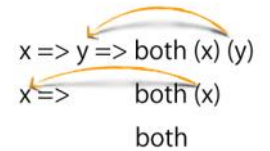
eta: reduction, $x \Rightarrow \text{foo}(x)$, eta reduces the unnecessary stuff to foo

-

Beta Reduktion II



Eta Reduktion II



L01 Types of String, Hoisted, Currying, Scopes, IIFE, Lambda Calculus, ...

`document.writeln("something");`: to write onto page

JavaScript functions:

- JavaScript doesn't need a Compiler
- 2 phase interpreter: first phase only locates space for variables and object, second phase initialises
- can't overload method, they get overwritten, "last one wins"
- no explicit return in functions returns undefined
- default parameter can be initialised in the method parameter list with e.g. `arg = 0`
- functions can be stored into `const`
- currying = transforms function with multiple arguments into a sequence of nesting functions
- hoisted: can be used before declaration like function, `const` is not hoisted
- three types of string: "it's time" and 'he said "hi"' and `x ist \${x}`

`String(/hdghsj/)`

`(typeof String = function)`

- two scopes:

global: alles zugreifbar (im browser: window)

function: variables are local to the enclosed function

`let x = ...` mutable, local scope

`const x = ...` immutable*, local scope

- IIFE: immediately invoked function expression

```
((() => {  
    let x = 1;  
    document.writeln(String(x));  
}))();
```

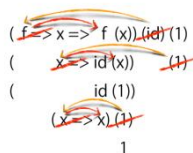
- Lambda Calculus

alpha: renames variables, $x \Rightarrow x$ is alpha equivalent to $y \Rightarrow y$

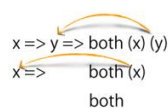
beta: runs function, $(x \Rightarrow x+1)(5)$, beta reduces to 6

eta: reduction, $x \Rightarrow \text{foo}(x)$, eta reduces the unnecessary stuff to `foo`

Beta Reduktion II



Eta Reduktion II



L02

- console.log("something")
- console.dir(foo) // for properties
- console.log(document.body) // possible
- console.warn or .info or .error (gives back stack back too)
- console.table(array) // doesnt show functions
- Baseline Widely available (on every major version of chrome, firefox, safari and edge)
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

- Kolibri
<https://github.com/WebEngineering-FHNW/Kolibri>

- CTRL K + CTRL U => uncomment code

- closure -> scope von variablen, von aussen kommt man nicht ran
- TODO: anleitung zum konstruieren von and etc mit lambda calculus
- product type
- sum type
- co-product

- console input: 1,2,3 returns 3 => expression concatenation
- partial application => map, filter, reduce
- reduce(function, 0) // use start value to not check for null arrays
- monoid
- array.every()
- array.some()

```
> [1,2,3,4,5].reduce( (acc, cur) => {acc.push(cur); return acc;}, [])  
< ▶ (5) [1, 2, 3, 4, 5]  
  
> [1,2,3,4,5].reduce( (acc, cur) => {acc.unshift(cur); return acc;}, [])  
< ▶ (5) [5, 4, 3, 2, 1]
```

- filter function with reduce
- fibonacci, function = memoize(function)

L02 Console., Kolibri, High Order Function, Product & Sum Type, Map, Filter, Reduce, Memoize

- console.log("something")
- console.dir(foo) // for properties
- console.log(document.body) // possible
- console.warn or .info or .error (gives back stack back too)
- console.table(array) // doesnt show functions
- Baseline Widely available (on every major version of chrome, firefox, safari and edge)
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
- Kolibri
<https://github.com/WebEngineering-FHNW/Kolibri>
- CTRL K + CTRL U => uncomment code
- High Order Functions: functions taking in functions as argument
- closure -> scope von variablen, von aussen kommt man nicht ran
- TODO: anleitung zum konstruieren von and etc mit lambda calculus
- product type
- sum type
- co-product
- console input: 1,2,3 returns 3 => expression concatenation
- partial application => map, filter, reduce
- reduce(function, 0) // use start value to not check for null arrays
- monoid
- array.every()
- array.some()
- ```
> [1,2,3,4,5].reduce((acc, cur) => {acc.push(cur); return acc;}, [])
< ▶ (5) [1, 2, 3, 4, 5]
```
- ```
> [1,2,3,4,5].reduce( (acc, cur) => {acc.unshift(cur); return acc;}, [])  
< ▶ (5) [5, 4, 3, 2, 1]
```
- filter function with reduce
- fibonacci, function = memoize(function)

L03 Scripting, PWA, Excel, Objects

const M = f => f(f)
const Y = M(M) | Y Komibinator
(f => f(f)) (f => f(f)) | Beta-Reduction
(f => f(f))(f => f(f))
=> Schleife, Selbstreplikation
Modus Ponens

Goodie

```
> const obj = {a:1, b:2};  
< undefined  
> obj.a  
< 1  
> obj.a = 2;  
< 2  
> obj  
< {a: 2, b: 2}  
> obj["a"]  
< 2  
> obj["a"] = 3  
< 3  
> obj  
< {a: 3, b: 2}  
> obj[1] = 3  
< 3  
> obj  
< {1: 3, a: 3, b: 2}  
> const arr = [];  
< undefined  
> arr[42] = 0  
< 0  
> arr  
< (43) [empty × 42, 0]  
> arr.1  
< undefined  
> arr.length  
< 43
```

literal vs. functional construcor
functions are objects in javascript

```
> arr.forEach( x => console.log(x));  
0  
< undefined  
> const b = Object();  
< undefined  
> Object.getOwnPropertyNames  
< f getOwnPropertyNames() { [native code] }  
> Object.getOwnPropertyNames()  
< ▶ Uncaught TypeError: Cannot convert undefined or null to object  
   at Object.getOwnPropertyNames (<anonymous>)  
   at <anonymous>:1:8  
> Object.getOwnPropertyNames(obj)  
< ▶ (3) ['1', 'a', 'b']
```

scripting: evaluating text
progressive web app: den code baut den man gerade baut

Parser understands, interpreter runs, script is what you write.

Why Scripting?

Command Line, Automation, Build System, Templating, Code Distribution, Formulae, Business Rules, Smart Configuration, Product Lines, DSL, **Self-Modifying Code**, ...

self-modifying code
ajegs?

scripting characteristics:

- interpreted not compiled
- lenient type system
- best effort approach

Google Maps Contraversy with javascript part they require you to get from them (script)

- `eval("x=3")` the same as `x=3` into console
- `cdl?`
- `Function()` = `eval()` but executes in global scope
e.g: `const add = Function('x', 'y', 'return x+y');`
`add(1,2);`
`add(2,5);` // no need to re-parse

Excel

```
> window.canvas
< <canvas id="canvas" width="800" height="600"> </canvas>
> window.user_function
< <input type="text" id="user_function" value="Math.sin(x)">
> |
```

window is the global "this" => `window.canvas` == `this.canvas` == `canvas`

scripting caution

Goodie Deonstruktor:

```
> const obj = {a:1, b:2};
< undefined
> const {b} = obj;
< undefined
> b
< 2
> let foo = ({a,b}) => console.log(a,b);
< undefined
> foo({a:1, b:3, c:5});
1 3
```

```
> const [g,h] = arr;
< undefined
> g
< 1
> h
< 2
> arr
< ▶ (3) [1, 2, 3]
```

another method to deconstructor:

```
> obj
< ▶ {a: 1, b: 2}
> obj...
✖ Uncaught SyntaxError: Unexpected token '...'
> {x:42, ...obj}
< ▶ {x: 42, a: 1, b: 2}
> {...obj}
< ▶ {a: 1, b: 2}
> |
```

- Tag typing
(... has different uses, here for deconstructor used)

```
> const arr = [1,2,3];
< undefined
> arr
< ▶ (3) [1, 2, 3]
> arr.toString()
< '1,2,3'
> [0, ...arr]
< ▶ (4) [0, 1, 2, 3]
```

reverse array:

```
> [0, ...arr]
< ▶ (4) [0, 1, 2, 3]
> arr.reduce( (acc, cur) => [cur, ...acc], [])
< ▶ (3) [3, 2, 1]
```



```

> const [g,h] = arr;
< undefined
> g
< 1
> h
< 2
> arr
< ▶ (3) [1, 2, 3]
> let bar = ([g,h]) => console.log(g,h);
< undefined
> bar([g,h]);
1 2
< undefined
> bar(arr);
1 2
< undefined
> bar = ([g,h, ...rest]) => console.log(g,h,rest);
< ([g,h, ...rest]) => console.log(g,h,rest)
> bar(arr)
1 2 ▶ [3]
< undefined

```

```

> bar(arr)
1 2 ▶ [3]
< undefined
> bar([1,2,3,4,5,6]);
1 2 ▶ (4) [3, 4, 5, 6]
< undefined
> const [e,r,...s]=[1,2,3,4,5,6];
< undefined
> e
< 1
> r
< 2
> s
< ▶ (4) [3, 4, 5, 6]
> const baz = (a,b,c) => console.log(a,b,c);
< undefined
> baz(arr);
▶ (3) [1, 2, 3] undefined undefined
< undefined
> baz(...arr);
1 2 3
< undefined

```

Objects:

- datastructures
- methods of access and management
- location for mutable state
- abstraction and polymorphism

Different Approaches

Open, dynamic

Closed, explicit

Mixed, classified

the same:

```
foo : function() { return this.a }  
foo() { return this.a }           // syntactic sugar
```

it was possible to redefine undefined (to a number or something) you cant tdo it anymore

- open, dynamic: const obj contains function as parameter
- closed explicit: function that returns new object, function starts with uppercase
- mixed classification: new keyword: new is bound by this.? TODO: look up
this is what happens with "class"

L04

Tuesday, January 20, 2026 10:22 AM

- function instead of const: when two methods are referencing each other
- class syntactic sugar to mixed classifies, there doesn't really exist a class, it's a function
- no implements, but extends exists, syntax sugar for prototype and saves some memory but can be a security issue because of the "this" compared to closed explicit variant, prototype chain => more runtime, runtime vs. memory tradeoff
- functions are objects, they have prototype property but objects aren't functions

```
> const n = 42;
< undefined
> n.toString()
< '42'
> Number.prototype.toString = function(){ return "1"; }
< f () { return "1"; }
> n.toString()
< '1'
```

(toString ist eine property)

PUT TIMES INTO TOOLBOX

JS GOODIE

```
> [1,2,3,4].map( it => it *2);
  (4) [2, 4, 6, 8]
> [1,2,3,4].map( (it,idx) => it *idx>2);
  (4) [false, false, true, true]
> [1,2,3,4].map( (it,idx) => it*idx > 0);
  (4) [false, true, true, true]
> [1,2,3,4].every( (it,idx) => it*idx > 0);
false
> [1,2,3,4].some( (it,idx) => it*idx > 0);
true
> [1,2,3,4].filter( (it,idx) => it*idx > 0);
  (3) [2, 3, 4]
> [0,1,2,3,4].slice(2,4);
  (2) [2, 3]
> [0,1,2,3,4].slice();
  (5) [0, 1, 2, 3, 4]
> [0,1,2,3,4].slice(-1);
  (1) [4]
> [0,1,2,3,4].slice(-3);
  (3) [2, 3, 4]
```

```

> [0,1,2,3,4].splice(1,3);
< ▶ (3) [1, 2, 3]
> const arr = [0,1,2,3,4];
< undefined
> arr.splice(1,3);
< ▶ (3) [1, 2, 3]
> arr
< ▶ (2) [0, 4]
> arr.splice(1,0,7,8,9);
< ▶ []
> arr
< ▶ (5) [0, 7, 8, 9, 4]

```

- use .reverse etc. won't change original object then

JsDoc

Untyped:

- built in type, not easily extendable, Literal constructs {} => ()
BigInt: 1n, 1n + 1 doesn't exist
typeof exists for these until here

Object "classes": Array, Date, JSON, ...

- @function, @callback
- @type

Data Types

- Objects: @property explains the property
- @typedef Interfaces

Frage: param vs. property?

L05

JS Goodie:

```
[1,2,3].forEach( it => { if(it > 1) return ; console.log(it) });
```

it++ wont change the it like in a for loop

```
document.querySelectorAll("a").forEach(it => {if(it > 1) return; console.log(it)});
```

search all elements on the website with "a" in it

=> same:

```
for( const i of document.querySelectorAll("a")) {console.log(i)};
```

```
for( const i in document.querySelectorAll("a")) {console.log(i)};
```

=> sucht alle properties

```
for( const i in [1,2,3]) {console.log(i)};
```

special case? rare case

Monad => promise to promise....

```
[1,2,3].flatMap(it => [2,3]);
```

flattens the list into one list

Functor, Monoid, Monad => important

Promise takes as an argument a function which takes two functions as argument itself

```
const processEven = i => new Promise( (resolve, reject) => {  
    if (i % 2 === 0) {  
        resolve(i);  
    } else {  
        reject(i);  
    }  
});
```

success/failure callbacks

resolve and reject depending on "gültige" promise

Modules: closed and dependencies are clear

Layers:

- Standard Library
- Utility Library
- andere Libraries

JS Goodie:

```

> const x = 1;
< undefined
> { x: x }
< {x: 1}
> { x }
< {x: 1}
>

```

modules are not

- packages they have versions
- objects
- package manager
- build tools

modules are:

- uses import or export

FunFact: URL vs. URI

import variant, use this: `import { export1 , export2 } from "module-name";`

Import Variants

always explicit

```

import "module-name";
import defaultExport from "module-name";
import * as name from "module-name";
import { export } from "module-name";
import { export as alias } from "module-name";
import { export1 , export2 } from "module-name";
const promise = import("module-name");

```

export variant, use this: `export { name1, name2, ..., nameN };`

in modules: no more `document.writeln`, no global stuff and no this

L06

JS Goodie:

- regular expression

```
> "test".split(/ /);
< ▶ ['test']
> "test teeeest".split(/ /);
< ▶ (2) ['test', 'teeeest']
> "test teeeest".split(/\b/);
< ▶ (3) ['test', ' ', 'teeeest']
> "test /t teeeest".split(/\b/);
< ▶ (5) ['test', ' ', 't', ' ', 'teeeest']
> "test /t teeeest".split(/\W/);
< ▶ (4) ['test', '', 't', 'teeeest']
> "test /t === teeeest".split(/\W/);
< ▶ (8) ['test', '', 't', '', '', '', 'teeeest']
> "test /t === teeeest".split(/\s?/);
< ▶ (16) ['t', 'e', 's', 't', '/', 't', '=', '=', 't', 'e', 'e', 'e', 'e', 's', 't']
> "test /t === teeeest".split(/\s+/);
< ▶ (4) ['test', '/t', '===', 'teeeest']
> "test /t === teeeest".split(/\s*/);
< ▶ (16) ['t', 'e', 's', 't', '/', 't', '=', '=', 't', 'e', 'e', 'e', 'e', 's', 't']
```

```
> "test teeeest".search(/\w+/);
< 0
> "test teeeest".search(/\w{7}/);
< 5
> "test teeeest".match(/\w*/);
< ▶ ['test', index: 0, input: 'test teeeest', groups: undefined]
> "test teeeest".matchAll(/\w+ \w+/g);
< ▶ RegExpStringIterator {}
> const res = "test teeeest".matchAll(/\w+ \w+/g);
< undefined
> [...res]
< ▶ [Array(1)]
> const re = "test teeeest".matchAll(/(\w+) (\w+)/g);
< undefined
> re
< ▶ RegExpStringIterator {}
> [...re]
< ▶ [Array(3)]
> "test teeeest".replaceAll(/(\w+) (\w+)/g, "___");
< '___'
> "test teeeest".replaceAll(/(\w+) (\w+)/g, "_,_");
< '_,_'
> "test teeeest".replaceAll(/(\w+) (\w+)/g, (full, g1, g2) => g2 + " " + g1);
< 'teeeest test'
```

```
"The rain in spain stays mainly in the plains".replaceAll(/(\w\w\w)(.*?)\1/g, (full, g1, fill, g2) => "___" + fill + "___");
```

types of asynchrony

- **no coordination**: all actions run independently
- delegated coordinated -> **Scheduler**
queue FIFO with locks, callback unlocks
- result dependency -> implicit coordination => **DataFlowVariable...**

is a function, lazy,

JS Goodie:

```
> Array.prototype["+"] = function (arr) { return [...this, ...arr] };  
< f (arr) { return [...this, ...arr] }  
> [1,2,3]["+"]([4,5,6]);  
< ▶ (6) [1, 2, 3, 4, 5, 6]
```

```
> Array.prototype.xxx = function (arr) { return [...this, arr] };  
< f (arr) { return [...this, arr] }  
> [1,2,3].xxx([4,5,6]);
```

Iterator Protocol

- destructors in parameter position
- decouple infinite production from lazy consumption

L07

JS Goodie: Zahlen Parser/Darstellung

```
> Math.PI | 0
< 3
> 2**3
< 8
> 1E3
< 1000
> 1E-3
< 0.001
> Intl.NumberFormat("en-us").format(1000.01);
< '1,000.01'
> Intl.NumberFormat("de-ch").format(1000.01);
< '1'000.01'
> Intl.NumberFormat("de-de").format(1000.01);
< '1.000,01'
> 1_000_000
< 1000000
```

06.02. Toolbox Abgabe per Mail

Moves 0-7

- 0 Explore & Timebox
- 1 Goal Card, state and stick to the goal and no interruptions
- 2 Start at the End: was ist das resultat das wir wollen in statischen/konkreten Werten und rückwärts anfangen => kann auch test case sein
Evtl. Go back to explore step if missing stuff
- 3 Extract: replace static values with variables
- 4 Abstract: pattern erkennen und vereinfachen (falls es einfacher wird)
- 5 Reorganize: refactor evtl. auslagern
- 6 release: tests, documentation, examples => before repo push
- 7 retro: what to keep and what to do differently

JS Goodie Nachmittag:

```
> Walk(1,10,2).show()
< ' [1,3,5,7,9] '
> Walk(1,Number.MAX_VALUE,2).show()
< ' [1,3,5,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,37,39,41,4
9,71,73,75,77,79,81,83,85,87,89,91,93,95,97,99] '
> Walk(ALL).take(5).show()
< ' [0,1,2,3,4] '
> Walk(ALL).take(5).map(n=>String(n)).show()
< ' [0,1,2,3,4] '
> Walk(ALL).map(n=>String(n)).take(5).show()
< ' [0,1,2,3,4] '
> Walk(ALL).map(String).take(5).show()
< ' [0,1,2,3,4] '
```

MVC:

L08

JS Goodie:

dont user parseInt but Number, s=> parseInt(s,10); Basis 10 ist Standard

```

> Number("8");
< 8
> parseInt("8"); // shouldn't be used
< 8
> ["3","4","5"].map( s => Number(s));
< ▶ (3) [3, 4, 5]
> ["3","4","5"].map(Number);
< ▶ (3) [3, 4, 5]
> ["3","4","5"].map(s => parseInt(s));
< ▶ (3) [3, 4, 5]
> ["3","4","5"].map(parseInt);
< ▶ (3) [3, NaN, NaN]

```

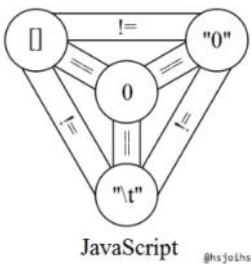
MVC: API, Controller, Model classic version

Kolibri: [Kolibri](#)

Nachmittag:

nicht Äquivalenzrelation: nicht transitiv und nicht identität

Trinity



```
assert.isTrue(!"0" == false);
```

turns !"0" : ! turns the "0" into a boolean so false

!!"0" turns "0" first into a boolean value false and then negates it again

+[] turns it into a number so 0

Rule of thumb (not reliable)

object -> string -> number -> boolean

Number.MAX_SAFE_INTEGER: nachdem ist die Berechnung nicht sicher

From <<https://teams.microsoft.com/v2/>>

$$\{\} \geq \{\},$$

$\geq$ wird ersetzt durch $<$ und dann negiert

Takeaway

prefer === over ==

remaining case:

(a == null) better than

```
(a === null || a === undefined)
```

[illegible]

```
> typeof NaN
< "number"
> 999999999999999999
```

```
> true==1
< true
> true===1
```

	true	false	1	0	-1	"true"	"false"	"1"	"0"	"-1"	""	null	undefined	Infinity	-Infinity	{}	{}	[]	[0]	[1]	NaN
true																					
false																					
1																					
0																					
-1																					
"true"																					
"false"																					
"1"																					
"0"																					
"-1"																					
""																					
null																					
undefined																					
Infinity																					
-Infinity																					
{}																					
{ }																					
[]																					
[0]																					
[1]																					
NaN																					

> typeof NaN	> true==1
< "number"	< true
> 9999999999999999	> true===1
< 10000000000000000	< false
> 0.5+0.1==0.6	> (!+[+][+][+]).length
< true	< 9
> 0.1+0.2==0.3	> 9+"1"
< false	< "91"
> Math.max()	> 91-"1"
< -Infinity	< 90
> Math.min()	> []==0
< Infinity	< true
> []+[]	
< ""	
> []+{ }	
< "[object Object]"	
> { }+[]	
< 0	
> true+true+true===3	
< true	
> true-true	
< 0	



Quiz <http://86.119.43.169:9090/>

Q10b

Assignment: Using 'Function()'

Attempt number 2 of at most 3 attempts

Your user has entered a new business rule on how to calculate this year's bonuses. He entered a formula through a web form and it is available through the `_string_` variable `'bonusFormula'`. The formula refers to the employee as `'employee'` like in `'employee.revenue * 0.2'` (note that this only calculates but does not set the bonus)

Provide a function (or lambda expression) named `'bonusCalculation'` that calculates an employee's bonus by using the user-provided `bonusFormula`.

Since we will give bonuses to many employees, do NOT use the `'eval'` function and do NOT use side effects like setting surrounding variables.

your code here, max 512 chars

Remaining chars: 512

Your solution will be tested against:

```
const e = {revenue: 10000, bonus: null};
bonusCalculation(e);
e.bonus === e.revenue * factor_;
```

```
const bonusCalculation = (employee) =>
  (employee.bonus = new Function(
    "employee",
    `return (${bonusFormula}) * employee.revenue`,
  )(employee));
```

Q34a

Assignment: Where JavaScript is untrue

Attempt number 1 of at most 3 attempts

provide code like
const value = ____;
where you replace the ____ placeholder with a value that will yield
different results when used in
if (value) ...
as opposed to
if (value == true)

your code here, max 512 chars

Remaining chars: 512

Your solution will be tested against:

```
;
(value ? true : false) !== (value == true ? true : false)
```

Q34b

Assignment: Where JavaScript is unordered

Attempt number 1 of at most 3 attempts

provide code like
const value = ____;
where you replace the ____ placeholder with a value that will yield
{ } == value === false
{ } > value === false
{ } >= value === true

your code here, max 512 chars

Remaining chars: 512

Your solution will be tested against:

```
;
(({ } == value) === false) && (({ } > value) === false) && (({ } >= value) === true)
```

Q33

Crazy JavaScript

The numerical value of true is 1. :



'typeof (+{})' is 'number'. :



'typeof (+{})' is 'number'. : ☒

if $a==b$ then follows that $a==!b$ must be false for all values a and b. : ☒

it is preferred to use `===` over `==`. : ☒

The boolean value of `{}` is false. : ☒

The boolean value of `0` is false. : ☒

The boolean value of `"0"` is false. : ☒

*Deconstructors allow to pick multiple property values from an object at once. : ☒

The boolean value of `[]` is false. : ☒

The boolean value of `""` is false. : ☒

if $a==b$ and $b==c$ then follows that $a==c$ for all values of a,b,c. : ☒

if $a==b$ is false and $a>b$ is false then $a>=b$ cannot possibly be false for any values a and b. : ☒

The numerical value of `{}` is NaN. : ☒

'if(x)' and 'if(x==true)' will always lead to the same behaviour. : ☒